



Arquitetura WhatsApp MCU Night Run

Camada de mensageria baseada em Cloudflare Worker, Cloudflare Tunnel, Evolution API, Redis, Neon Postgres e Firebase. Inclui topologia, fluxos de envio, persistencia, operacao, seguranca e runbook.



Versao	Data	Ambiente	Escopo
1.0	15/05/2026	Producao operacional	WhatsApp, Worker, Tunnel, Redis, Neon, Firebase

1. Sumario Executivo

A arquitetura WhatsApp do MCU Night Run separa o frontend publico, a API de aplicacao e o motor de mensageria. O aplicativo React hospedado no Firebase chama um Cloudflare Worker. O Worker atua como camada de seguranca, proxy, fila e orquestrador. A entrega das mensagens e feita por uma Evolution API privada, publicada apenas por Cloudflare Tunnel em **whatsapp.mcunightrun.com.br**.

3 containers em execucao	1 instancia WhatsApp	1 min ciclo de fila cron	0 segredos no PDF
------------------------------------	--------------------------------	------------------------------------	-----------------------------

Resultado atual

A infraestrutura esta publicada e acessivel via Cloudflare Tunnel. A instancia **mcu_nightrun_uba** existe; quando estiver desconectada, basta gerar o QR pelo endpoint **/whatsapp/connect** e parear o aparelho.

Principios de desenho

- **Fronteira publica unica:** o app nao fala diretamente com a Evolution API; todo trafego passa pelo Worker.
- **Origem protegida:** a Evolution escuta localmente em 127.0.0.1:8080 e e exposta externamente apenas pelo Tunnel.
- **Estado separado:** mensagens e logs operacionais ficam em Firestore/KV; metadados internos da Evolution ficam no Neon.
- **Operacao recuperavel:** Docker Compose reinicia containers; Redis usa AOF; Neon persiste o banco gerenciado.

2. Visao Geral da Topologia

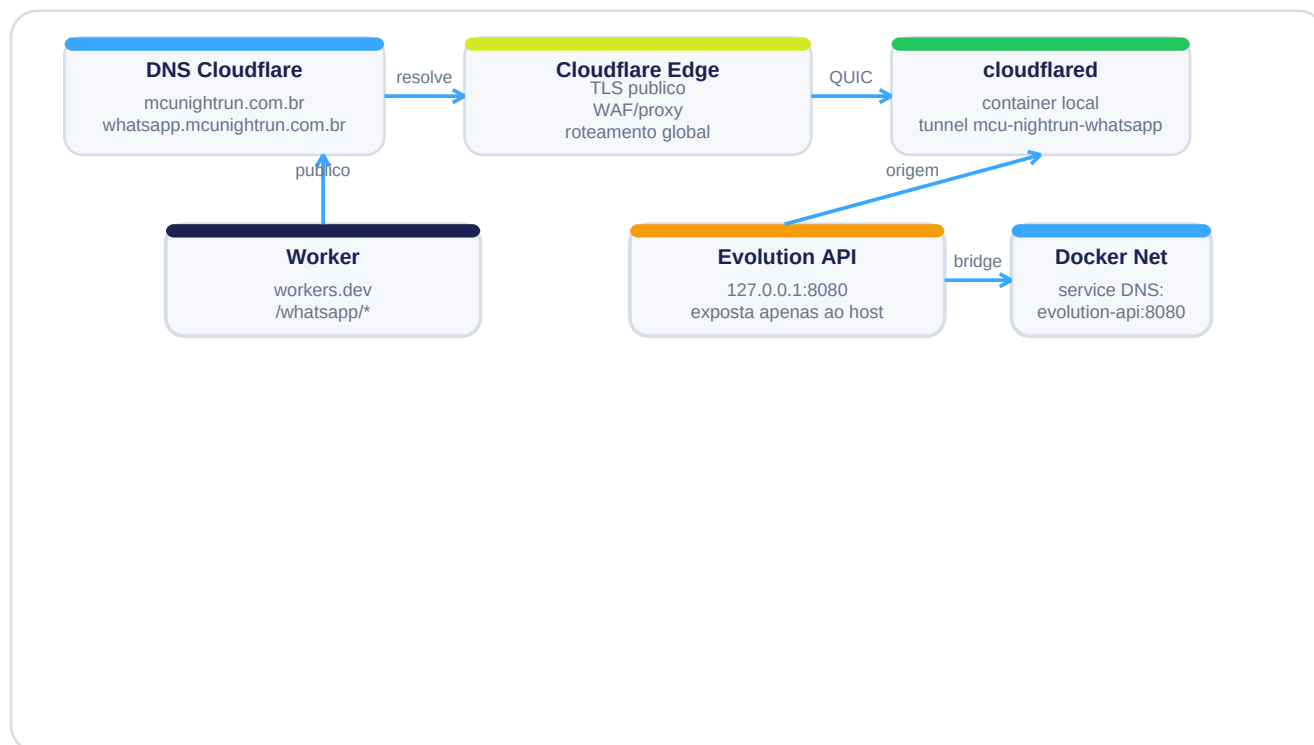
Diagrama de componentes principais



O Worker concentra as regras de negocio que conectam inscicoes, pagamentos e mensagens. A Evolution API fica dedicada ao protocolo WhatsApp/Baileys, reduzindo o acoplamento entre o sistema do evento e a sessao do WhatsApp.

Camada	Componente	Responsabilidade	Estado mantido
Frontend	Firebase Hosting + React	Cadastro, painel admin, chamadas HTTP para Worker	Estado de UI e dados Firestore
API	Cloudflare Worker	Proxy, fila, cron, upload R2, Asaas, WhatsApp	KV para fila e flag de pausa
Mensageria	Evolution API v2.3.7	Sessao WhatsApp, QR, envio texto/midia	Instancias e mensagens no Neon
Persistencia	Neon Postgres	Banco da Evolution e migracoes Prisma	Tabelas internas da Evolution
Cache	Redis 7	Cache e estado rapido da Evolution	Chaves Redis com AOF
Rede	Cloudflare Tunnel	Publicacao segura da API privada	Conexoes QUIC ativas

3. Rede, DNS e Exposicao Publica



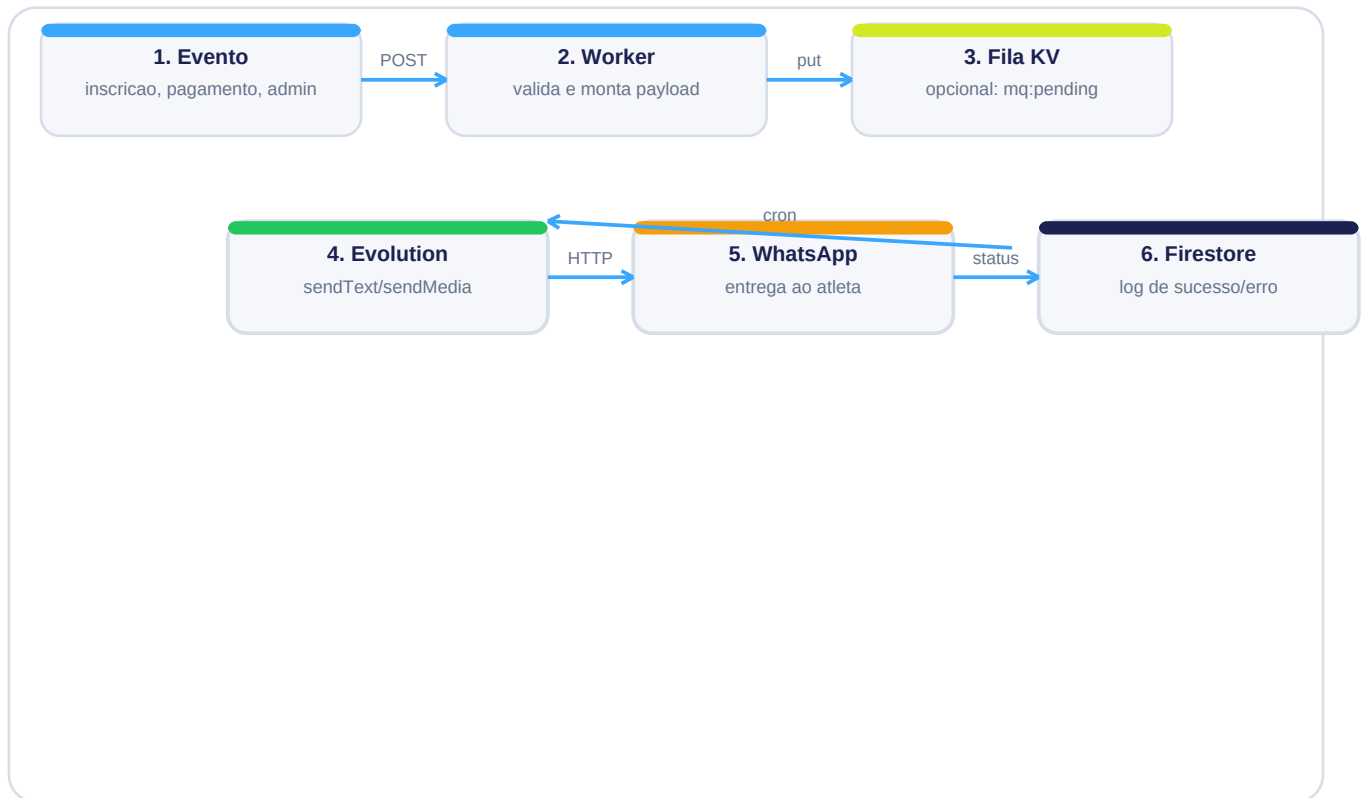
O domínio **mcunightrun.com.br** passou a ser gerenciado por Cloudflare DNS. O site principal continua apontando para Firebase Hosting via registro A **199.36.158.100**. O subdomínio **whatsapp.mcunightrun.com.br** é roteado pelo tunnel **mcu-nightrun-whatsapp** para o serviço interno **http://evolution-api:8080** na rede Docker.

Host	Destino	Proxy	Uso
mcunightrun.com.br	199.36.158.100	DNS only recomendado para validacao Firebase	Site publico Firebase Hosting
www.mcunightrun.com.br	mcunightrun.com.br	DNS only	Alias do site principal
whatsapp.mcunightrun.com.br	Cloudflare Tunnel	Proxy Cloudflare	Origem publica segura da Evolution API

Importante sobre origem

A porta Docker publicada é **127.0.0.1:8080**, não 0.0.0.0. Isso impede exposição direta pela rede local. O acesso público ocorre pela conexão outbound do cloudflared.

4. Fluxo de Envio de Mensagens



Caminhos principais

- **Boas-vindas / PIX:** apos a inscricao, o PublicForm chama o Worker em **/whatsapp/send** com texto e imagem opcional.
- **Pagamento confirmado:** o webhook Asaas atualiza Firestore e enfileira uma confirmacao em **queueMessage**.
- **Sorteio surpresa:** se uma regra de sorteio for acionada, o Worker registra o premio e enfileira mensagem especifica.
- **Envios administrativos:** paginas admin usam o Worker para disparos diretos, historico e fila.

Para mensagens com imagem, o Worker baixa a midia por URL, converte para base64 e envia pela rota **/message/sendMedia/{INSTANCE_NAME}**. Mensagens sem midia usam **/message/sendText/{INSTANCE_NAME}**. O payload inclui delay e presence composing para reduzir comportamento brusco de envio.

5. Fila, Cron e Controle de Ritmo



A fila do Worker usa Cloudflare KV como buffer leve. Cada mensagem pendente recebe chave no formato **mq:pending:{timestamp}:{random}**. O cron roda a cada minuto, consulta ate 15 chaves por ciclo, envia pela Evolution e remove a chave somente apos resultado tratado.

Elemento	Valor / comportamento	Finalidade operacional
Prefixo pendente	mq:pending:*	Localizar mensagens ainda nao processadas
Flag de pausa	mq:paused	Interromper disparos sem derrubar infraestrutura
Lote por ciclo	15 mensagens	Limitar rajadas e controlar consumo
Intervalo entre envios	4 segundos	Reduzir risco de bloqueio por disparo agressivo
Log	Firestore whatsapp_logs	Auditar sucesso, erro, destinatario e tipo

Limite de desenho

Cloudflare KV e suficiente para fila simples, mas nao entrega semantica forte de fila transacional. Para volumes maiores, recomenda-se migrar para Cloudflare Queues, Durable Objects ou uma fila dedicada.

6. Contratos de API do Worker

Endpoint	Metodo	Destino interno	Uso
/whatsapp/status	GET	/instance/connectionState/{INSTANCE_NAME}	Verificar estado open, connecting ou close
/whatsapp/create	POST	/instance/create	Criar instancia Baileys com QR habilitado
/whatsapp/connect	GET	/instance/connect/{INSTANCE_NAME}	Gerar QR ou reconectar instancia
/whatsapp/logout	POST	/instance/logout/{INSTANCE_NAME}	Encerrar sessao atual
/whatsapp/send	POST	/message/sendText ou sendMedia	Enviar texto ou midia para numero normalizado
/queue/process	POST	processQueue(env)	Forcar processamento manual da fila
/queue/toggle-pause	POST	KV mq:paused	Pausar ou retomar fila

Contrato de envio simplificado

```
{
  "phone": "5532999999999",
  "text": "Mensagem ao atleta",
  "imageUrl": "https://.../imagem.png" // opcional
}
```

O Worker e responsavel por adicionar cabecalhos **apikey**, selecionar endpoint de texto ou midia e encapsular resposta como **{ success, status, response }**.

7. Persistencia e Donos dos Dados

Dado	Fonte da verdade	Consumidores	Retencao / observacao
Inscricao e pagamento	Firestore nightrun_registrations	App, Worker, admin	Dados principais do atleta e status de pagamento
Logs WhatsApp	Firestore whatsapp_logs	Admin mensagens, auditoria	Resumo de destinatario, status, erro e tipo
Fila pendente	Cloudflare KV NIGHTRUN_STORAGE	Worker cron	Ephemeral ate envio com sucesso
Midias uploadadas	Cloudflare R2 MEDIA_BUCKET	App e mensagens	Objetos de imagem e anexos
Estado Evolution	Neon Postgres	Evolution API	Instancias, chats, mensagens, contatos e configuracoes internas
Cache Evolution	Redis	Evolution API	Estado rapido e cache de instancia

A separacao e intencional: o sistema do evento nao depende de detalhes internos da Evolution. Se a Evolution for trocada no futuro, o contrato a preservar e o Worker, nao o frontend.

8. Ciclo de Vida da Instancia WhatsApp

Estado	Significado	Acao recomendada
close	Sessao fechada ou nao pareada	Chamar /whatsapp/connect e escanear QR
connecting	QR emitido ou tentativa de conexao em andamento	Aguardar pareamento; se expirar, gerar novo QR
open	WhatsApp conectado	Liberar disparos; monitorar logs e fila

Procedimento de pareamento

- Acesse **/whatsapp/status** pelo Worker para confirmar o estado.
- Se estiver **close**, acesse **/whatsapp/connect** para obter o QR.
- No celular do WhatsApp, use **Aparelhos conectados** e escaneie o QR.
- Repita **/whatsapp/status** ate retornar **open**.

Cuidado operacional

Nao compartilhe QR em canais publicos. Ele concede acesso a uma sessao WhatsApp. Gere o QR apenas quando alguem autorizado estiver pronto para parear.

9. Seguranca e Segredos

A arquitetura ja possui uma boa fronteira de seguranca: o frontend nao carrega a chave da Evolution; a API privada nao fica exposta diretamente; o tunnel usa conexao outbound; e a origem local fica restrita ao host. Ainda assim, ha medidas importantes para endurecer o ambiente.

Tema	Estado atual	Recomendacao
Evolution API key	Usada no Worker e no .env local	Migrar de wrangler.toml para wrangler secret
Cloudflare Tunnel token	Guardado no .env privado	Rotacionar se for exposto em conversa ou print
Neon connection string	Guardada no .env privado	Usar role dedicada e rotacionar se exposta
Firebase API key	Publica por natureza, mas deve ter regras Firestore fortes	Revisar regras e App Check se necessario
Worker CORS	Allow-Origin amplo	Restringir em producao para dominios do app
WhatsApp QR	Gerado sob demanda	Tratar como segredo temporario

Prioridade alta

Mover chaves de **wrangler.toml** para **wrangler secret**. O TOML e arquivo de configuracao versionavel; segredos devem viver no cofre da Cloudflare.

10. Operacao, Monitoramento e Diagnostico

Comandos essenciais

```
cd infra/evolution
docker compose ps
docker logs -f nightrun_evolution_api
docker logs -f nightrun_evolution_tunnel
docker compose up -d redis evolution-api cloudflared
```

Sintoma	Verificacao	Causa provavel	Acao
Worker retorna close	/whatsapp/status	Sessao desconectada	Gerar QR em /whatsapp/connect
URL whatsapp nao abre	docker logs tunnel	Rota ou tunnel fora	Revisar Public Hostname e cloudflared
Evolution reiniciando	docker logs api	Erro de banco/env	Validar .env e Neon
Mensagens paradas	/queue/toggle-pause e KV	Fila pausada ou erro de envio	Retomar fila e checar logs
Imagem nao envia	response Evolution	Download/base64/mimetype	Testar URL publica e tamanho da midia

11. Deploy e Mudancas Controladas

- **Infra Evolution:** alterar `infra/evolution/.env` e executar `docker compose up -d`.
- **Worker:** alterar configuracao/codigo e publicar com `npx wrangler deploy`.
- **DNS/Tunnel:** mudar rotas no painel Cloudflare Zero Trust; confirmar logs do cloudflared.
- **Rollback:** manter o compose e o Worker atual como unidade de mudanca; em falha, voltar TOML/codigo anterior e redeploy.

Mudanca	Risco	Janela recomendada	Teste de aceite
Trocar API key Evolution	Worker perde acesso se chave divergir	Baixo trafego	/whatsapp/status retorna 200
Trocar hostname do tunnel	Worker aponta para origem errada	Baixo trafego	GET hostname retorna Welcome Evolution
Atualizar Evolution image	Migracoes e compatibilidade	Janela controlada	Boot, status, envio teste
Alterar fila	Duplicidade ou perda de mensagens	Com fila pausada	Mensagem teste e log Firestore

12. Evolucao Recomendada

Prioridade	Melhoria	Beneficio
Alta	Migrar segredos para wrangler secret	Reduz risco de vazamento em arquivos versionados
Alta	Restringir CORS aos dominios oficiais	Diminui superficie publica do Worker
Media	Adicionar endpoint health consolidado	Diagnostico rapido de Worker, Tunnel, Evolution e instancia
Media	Mover fila para Cloudflare Queues	Melhor semantica de entrega e retry
Media	Alertas Cloudflare Tunnel e Docker	Reduz tempo de deteccao de queda
Baixa	Dashboard tecnico de WhatsApp	Visibilidade para administracao sem acessar logs

A arquitetura atual já está funcional para operação do evento. As recomendações acima não bloqueiam o uso, mas aumentam maturidade, governança e segurança antes de campanhas de maior volume.

13. Inventario Tecnico

Item	Valor
Worker	mcu-nightrun-api
Worker publico	https://mcu-nightrun-api.thayrufino2.workers.dev
Evolution publica	https://whatsapp.mcunightrun.com.br
Instancia	mcu_nightrun_uba
Tunnel	mcu-nightrun-whatsapp
Containers	nightrun_evolution_api, nightrun_evolution_redis, nightrun_evolution_tunnel
Neon	Projeto dedicado para Evolution, banco neondb, branch main
KV	NIGHTRUN_STORAGE
R2	MEDIA_BUCKET / mcu-nightrun-media
Firestore logs	whatsapp_logs

Escopo de confidencialidade

Este documento descreve a arquitetura e operacao, mas omite API keys, tokens Cloudflare, connection strings Neon, chaves Asaas e segredos de webhook.